NeuroVision: An Interactive Platform for Building, Training, and Inspecting Neural Networks with Measured Values in Synchronized 2D and 3D Views

K. Vishwa

Department of Computer Science and Engineering, Mahatma Gandhi Institute of Technology (A), Hyderabad – 500075, Telangana, India

https://github.com/vishwa-10147/neurovision

ABSTRACT

Neural-network visualization tools tend to fall into two groups. Educational playgrounds let learners build and train toy networks but cannot open a real model; model viewers show the graph of a real model but not what it computes. This paper presents NeuroVision, an open web platform (React/TypeScript frontend; FastAPI, PyTorch and ONNX Runtime backend) that combines both experiences on one normalized graph contract. Users can design dense networks backed by seeded PyTorch models and train them in background jobs whose loss, accuracy and activations stream over a replayable WebSocket. They can import trained ONNX models, run them in an isolated, time-limited ONNX Runtime worker, and inspect per-operator outputs, CNN feature maps, convolution filters and attention heads. They can also stream metrics and layer activations from their own training scripts through a one-file, standard-library Python client. A health monitor turns measured gradient norms, update ratios and dead or saturated units into alerts that quote the numbers behind them, and scripts that opt in can be paused, resumed, stopped, checkpointed or given a new learning rate from the dashboard.

Three principles govern the design: every value shown is measured rather than animated, every payload is bounded, and every uploaded model or incoming stream is untrusted. Synchronized 2D (SVG) and 3D (instanced WebGL) renderers share a semantic selection state and degrade to aggregate views beyond explicit detail limits. On a CPU laptop, educational networks reach 100% validation accuracy on two-moons and circles in about 2.5 s; a PyTorch CNN exported to ONNX and run in the worker reproduces PyTorch's outputs to within 5.2×10^{-8}; attention is recovered by pattern from a real *nn.MultiheadAttention* export; and warm inference inside the worker takes 0.06–0.21 ms on the sample models.

Keywords—neural-network visualization, explainable AI, ONNX, PyTorch, live training monitoring, interactive machine-learning education, attention visualization, feature maps, WebGL, sandboxed inference.